

Errores de descompilacion

Jose E Rodriguez Rios

Comparación entre código alto nivel, disassembly y descompilación para los programas `25.function.power.c`, `8.qsort.c` y `32.expr.c`, resaltando "errores" cometidos por el descompilador.

Código de `25.function.power.c`, `8.qsort.c` y `32.expr.c` disponible en <https://github.com/Heatwave/The-C-Programming-Language-2nd-Edition>

25.function.power

```
Undefined8 main(void)
```

```
{
    ulong uVar1;
    ulong uVar2;
    uint local_1c;

    local_1c = 0;
    while ((int)local_1c < 10) {
        uVar1 = power(-3,local_1c);
        uVar2 = power(2,local_1c);
        printf("%d %d %d\n", (ulong)local_1c,uVar2 & 0xffffffff,uVar1 & 0xffffffff);
        local_1c = local_1c + 1;
    }
    return 0;
}
```

En ghidra esta tratando uVar1 y uVar2 como una variable tipo ulong hay que cambiarlo a int para que no salga el bitwise operaciones.

Local_1c debe ser int.

Tambien el descompilador en ghidra guardo los resultados de power() en un variable temporero Uvar1 y Uvar2 en vez que en el high level se llama directamente en el print.

```
main()
{
    int i;

    for (i = 0; i < 10; ++i)
        printf("%d %d %d\n", i, power(2, i), power(-3, i));
    return 0;
}
```

```
undefined8 main(void)
```

```
{  
    ulong uVar1;  
    ulong uVar2;  
    uint local_1c;  
  
    local_1c = 0;  
    while ((int)local_1c < 10) {  
        uVar1 = power(-3,local_1c);  
        uVar2 = power(2,local_1c);  
        printf("%d %d %d\n", (ulong)local_1c,uVar2 & 0xffffffff,uVar1 & 0xffffffff);  
        local_1c = local_1c + 1;  
    }  
    return 0;  
}
```

```
main()
```

```
{  
    int i;  
  
    for (i = 0; i < 10; ++i)  
        printf("%d %d %d\n", i, power(2, i), power(-3, i));  
    return 0;  
}
```

El tipo de return no concuerda con el high level return. Se puede deducir como es el main va a devolver un int.

```
undefined8 main(void)
```

```
{
    ulong uVar1;
    ulong uVar2;
    uint local_1c;

    local_1c = 0;
    while ((int)local_1c < 10) {
        uVar1 = power(-3,local_1c);
        uVar2 = power(2,local_1c);
        printf("%d %d %d\n", (ulong)local_1c,uVar2 & 0xffffffff,uVar1 & 0xffffffff);
        local_1c = local_1c + 1;
    }
    return 0;
}
```

El loop while loop no concuerda con el high level loop que es un for pero tiene la misma función.

```
main()
```

```
{
    int i;

    for (i = 0; i < 10; ++i)
        printf("%d %d %d\n", i, power(2, i), power(-3, i));
    return 0;
}
```

functionPower main() arreglado

```
int main(void)
{
    int local_EAX_28;
    int local_EAX_45;
    int local_1c;

    local_1c = 0;
    while (local_1c < 10) {
        local_EAX_28 = power(-3, local_1c);
        local_EAX_45 = power(2, local_1c);
        printf("%d %d %d\n", (ulong)(uint)local_1c, (ulong)(uint)local_EAX_45, (ulong)(uint)local_EAX_28);
        local_1c = local_1c + 1;
    }
    return 0;
}
```

```
ulong power(int param_1,int param_2)
```

```
{  
    uint local_10;  
    int local_c;  
  
    local_10 = 1;  
    local_c = 1;  
    while (local_c <= param_2) {  
        local_10 = local_10 * param_1;  
        local_c = local_c + 1;  
    }  
    return (ulong)local_10;  
}
```

Los errores que se ve en esta función es que ghidra clasifico el return type y el tipo de numero que es local_10. Power se supone que devuelve un número tipo int para eso local_10 tiene que También ser int .

```
int power(int base, int n)
```

```
{  
    int i, p;
```

```
    p = 1;  
    for (i = 1; i <= n; ++i)  
        p = p * base;  
    return p;  
}
```

functionPower power() arreglado

```
int power(int param_1,int param_2)
{
    int int;
    int local_c;
    |
    int = 1;
    local_c = 1;
    while (local_c <= param_2) {
        int = int * param_1;
        local_c = local_c + 1;
    }
    return int;
}
```

qsort

```
undefined8 main(void)
```

```
{  
    __compar_fn_t in_RCX;  
    undefined4 local_28;  
    undefined4 local_24;  
    undefined4 local_20;  
    undefined4 local_1c;  
    undefined4 local_18;  
    undefined4 local_14;  
    undefined4 local_10;
```

```
    local_28 = 4;  
    local_24 = 3;  
    local_20 = 1;  
    local_1c = 7;  
    local_18 = 9;  
    local_14 = 6;  
    local_10 = 2;
```

```
    printArr((long)&local_28,7);  
    qsort(&local_28,0,6,in_RCX);  
    printArr((long)&local_28,7);  
    return 0;  
}
```

```
int main()
```

```
{
```

```
    int i;
```

```
    int v[ARR_LEN] = { 4, 3, 1, 7, 9, 6, 2 };
```

```
    printArr(v, ARR_LEN);
```

```
    qsort(v, 0, ARR_LEN-1);
```

```
    printArr(v, ARR_LEN);
```

```
    return 0;
```

```
}
```

En ghidra esta tratando el array v como 7 diferentes variables en vez de tratarlo como uno solo. La razón por esto es por que cree que cada entero en el arreglo es una diferente variable y no sabe los tipos de cada uno de esos variables.

```
undefined8 main(void)
```

```
{  
    __compar_fn_t in_RCX;  
    undefined4 local_28;  
    undefined4 local_24;  
    undefined4 local_20;  
    undefined4 local_1c;  
    undefined4 local_18;  
    undefined4 local_14;  
    undefined4 local_10;  
  
    local_28 = 4;  
    local_24 = 3;  
    local_20 = 1;  
    local_1c = 7;  
    local_18 = 9;  
    local_14 = 6;  
    local_10 = 2;  
    printArr((long)&local_28,7);  
    qsort(&local_28,0,6,in_RCX);  
    printArr((long)&local_28,7);  
    return 0;  
}
```

```
int main()  
{  
    int i;  
    int v[ARR_LEN] = { 4, 3, 1, 7, 9, 6, 2 };  
    printArr(v, ARR_LEN);  
    qsort(v, 0, ARR_LEN-1);  
    printArr(v, ARR_LEN);  
    return 0;  
}
```

En qsort ghidra cree que recibe 4 parámetros en vez que 3. Causando el nueva variable que no existía en el programa originar in_rcx

Qsort main() medio arreglado

```
int main(void)
{
    int *local_28;

    local_28._0_4_ = 4;
    local_28._4_4_ = 3;
    printArr((int *)&local_28,7);
    qsort((int *)&local_28,0,6);
    printArr((int *)&local_28,7);
    return 0;
}
```

Por lo que veo
hay un bug y no
se ve bien todo
los elementos
en el array.

```
void printArr(long param_1,int param_2)
```

```
{  
    int local_c;  
  
    local_c = 0;  
    while (local_c < param_2) {  
        printf("%d ",(ulong)*(uint*)(param_1 + (long)local_c * 4));  
        local_c = local_c + 1;  
    }  
    putchar(10);  
    return;  
}
```

```
void printArr(int v[], int len)
```

```
{  
    int i;  
    for (i = 0; i < len; i++)  
        printf("%d ", v[i]);  
    printf("\n");  
}
```

Param_1 debería ser un int array para que se entienda mejor el printf que esta llamado después.

Qsort printArray() arreglado

```
void printArr(int *param_1,int param_2)
{
    int local_c;

    local_c = 0;
    while (local_c < param_2) {
        printf("%d ",(ulong)(uint)param_1[local_c]);
        local_c = local_c + 1;
    }
    putchar(10);
    return;
}
```

```

void qsort(void *__base, size_t __nmemb, size_t __size, __compar_fn_t __compar)
{
    int iVar1;
    int iVar2;
    int local_10;
    int local_c;

    iVar2 = (int) __nmemb;
    iVar1 = (int) __size;
    if (iVar2 < iVar1) {
        swap((long) __base, iVar2, (iVar1 + iVar2) / 2);
        local_10 = iVar2;
        local_c = iVar2;
        while (local_c = local_c + 1, local_c <= iVar1) {
            if (*(int *)((long) __base + (long) local_c * 4) < *(int *)((long) __base + (long) iVar2 * 4)) {
                local_10 = local_10 + 1;
                swap((long) __base, local_10, local_c);
            }
        }
        swap((long) __base, iVar2, local_10);
        qsort(__base, __nmemb & 0xffffffff, (ulong)(local_10 - 1), (__compar_fn_t)(__nmemb & 0xffffffff));
        qsort(__base, (ulong)(local_10 + 1U), __size & 0xffffffff, (__compar_fn_t)(ulong)(local_10 + 1U));
    }
    return;
}

```

Podemos ver que en ghidra se esta interpretando que hay 4 parámetros en la función y en el high-level hay 3 parámetros.

```

void qsort(int v[], int left, int right)
{
    int i, last;
    void swap(int v[], int i, int j);

    if (left >= right)
        return;
    swap(v, left, (left + right) / 2);
    last = left;
    for (i = left+1; i <= right; i++)
        if (v[i] < v[left])
            swap(v, ++last, i);
    swap(v, left, last);
    qsort(v, left, last-1);
    qsort(v, last+1, right);
}

```

También se ve que pone todos los parámetros como el tipo de variable incorrectamente. `__base` se supone que sea el array tipo `int`, `nmemb` y `size` se supone que sea `int`.

Qsort qsort() arreglado

```
void qsort(int *v,int left,int right)

{
    int local_10;
    int local_c;

    if (left < right) {
        swap(v,left,(right + left) / 2);
        local_c = left + 1;
        local_10 = left;
        while (local_c <= right) {
            if (v[local_c] < v[left]) {
                local_10 = local_10 + 1;
                swap(v,local_10,local_c);
            }
            local_c = local_c + 1;
        }
        swap(v,left,local_10);
        qsort(v,left,local_10 + -1);
        qsort(v,local_10 + 1,right);
    }
    return;
}
```

```
void swap(long param_1,int param_2,int param_3)
```

```
{  
    undefined4 uVar1;  
  
    uVar1 = *(undefined4 *) (param_1 + (long)param_2 * 4);  
    *(undefined4 *) (param_1 + (long)param_2 * 4) = *(undefined4 *) (param_1 + (long)param_3 * 4);  
    *(undefined4 *) ((long)param_3 * 4 + param_1) = uVar1;  
    return;  
}
```

```
void swap(int v[], int i, int j)
```

```
{  
    int temp;  
  
    temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

Ghidra confunde el tipo de variable de param_1. Si se cambia el tipo de param_1 a un array tipo int entonces se va a descompilar correctamente.

```

void swap(long param_1,int param_2,int param_3)
{
    undefined4 uVar1;

    uVar1 = *(undefined4 *) (param_1 + (long)param_2 * 4);
    *(undefined4 *) (param_1 + (long)param_2 * 4) = *(undefined4 *) (param_1 + (long)param_3 * 4);
    *(undefined4 *) ((long)param_3 * 4 + param_1) = uVar1;
    return;
}

```

```

void swap(int v[], int i, int j)
{
    int temp;

    temp = v[i];
    v[i] = v[j];
    v[j] = temp;
}

```

Ghidra no pudo entender Uvar1 que debería ser int de 4 bits por que undefined4 usualmente dice que es un variable de 4 bits..

Qsort swap() arreglado

```
void swap(int *param_1,int param_2,int param_3)
{
    int temp_5f7bce56a3;

    temp_5f7bce56a3 = param_1[param_2];
    param_1[param_2] = param_1[param_3];
    param_1[param_3] = temp_5f7bce56a3;
    return;
}
```

expr

```

{
    undefined8 uVar1;
    ulong uVar2;
    char *pcVar3;
    double extraout_XMM0_Qa;
    double extraout_XMM0_Qa_00;
    double extraout_XMM0_Qa_01;
    double extraout_XMM0_Qa_02;
    double extraout_XMM0_Qa_03;
    double extraout_XMM0_Qa_04;
    double extraout_XMM0_Qa_05;
    double extraout_XMM0_Qa_06;
    char **local_98;
    int local_8c;
    char local_88 [112];
    double local_18;
    int local_c;

    local_98 = param_2;
    local_8c = param_1;
    if (param_1 == 1) {
        puts("Usage: expr 1 1 +");
        uVar1 = 1;
    }
}

```

```

main(int argc, char **argv)
{
    int type;
    double op2;
    char s[MAXOP];

    if (argc == 1) {
        printf("Usage: expr 1 1 +\n");
        return 1;
    }
}

```

Ghidra no pudo entender Uvar1 que es int en realidad y confundió uVar1 con un ulong y debería ser double.

```
void __cdecl main(int param_1, char param_2,
```

```
{  
    undefined8 uVar1;  
    ulong uVar2;  
    char *pcVar3;  
    double extraout_XMM0_Qa;  
    double extraout_XMM0_Qa_00;  
    double extraout_XMM0_Qa_01;  
    double extraout_XMM0_Qa_02;  
    double extraout_XMM0_Qa_03;  
    double extraout_XMM0_Qa_04;  
    double extraout_XMM0_Qa_05;  
    double extraout_XMM0_Qa_06;  
    char **local_98;  
    int local_8c;  
    char local_88 [112];  
    double local_18;  
    int local_c;  
  
    local_98 = param_2;  
    local_8c = param_1;  
    if (param_1 == 1) {  
        puts("Usage: expr 1 1 +");  
        uVar1 = 1;  
    }  
}
```

```
main(int argc, char **argv)  
{  
    int type;  
    double op2;  
    char s[MAXOP];  
  
    if (argc == 1) {  
        printf("Usage: expr 1 1 +\n");  
        return 1;  
    }  
}
```

En Ghidra el main no esta definido como es
en C siempre devuelve int.

```
undefined8 main(int param_1,char **param_2)
```

```
{
undefined8 uVar1;
ulong uVar2;
char *pcVar3;
double extraout_XMMQ_Qa;
double extraout_XMMQ_Qa_00;
double extraout_XMMQ_Qa_01;
double extraout_XMMQ_Qa_02;
double extraout_XMMQ_Qa_03;
double extraout_XMMQ_Qa_04;
double extraout_XMMQ_Qa_05;
double extraout_XMMQ_Qa_06;
char **local_98;
int local_8c;
char local_88 [112];
double local_18;
int local_c;
}
```

```
local_98 = param_2;
local_8c = param_1;
if (param_1 == 1) {
    puts("Usage: expr 1 1 +");
    uVar1 = 1;
}
```

```
001006ca 55          PUSH     RBP
001006cb 48 89 e5    MOV      RBP,RSP
001006ce 48 81 ec    SUB      RSP,0xa0
a0 00 00 00
```

```
main(int argc, char **argv)
```

```
{
```

```
    int type;
```

```
    double op2;
```

```
    char s[MAXOP];
```

```
    if (argc == 1) {
```

```
        printf("Usage: expr 1 1 +\n");
```

```
        return 1;
```

```
    }
```

Char[MAXOP] es igual a char[100] pero en ghidra se guarda en el stack 0xa0 o 112 en entero.

```

while( true ) {
    local_98 = local_98 + 1;
    local_8c = local_8c + -1;
    uVar2 = gettop(local_88,local_8c,local_98);
    local_c = (int)uVar2;
    if (local_c == -1) break;
    switch(local_c) {
case 0x2a:
    pop();
    pop();
    push((char *) (extraout_XMM0_Qa_02 * extraout_XMM0_Qa_01));
    break;
case 0x2b:
    pop();
    pop();
    push((char *) (extraout_XMM0_Qa_00 + extraout_XMM0_Qa));
    break;
default:
    printf("error: unknown command %s\n");
    break;
case 0x2d:
    pop();
    local_18 = extraout_XMM0_Qa_03;
    pop();
    push((char *) (extraout_XMM0_Qa_04 - local_18));
    break;
case 0x2f:
    pop();
    if (extraout_XMM0_Qa_05 == 0.00000000) {
        puts("error: zero divisor");
    }
    else {
        local_18 = extraout_XMM0_Qa_05;
        pop();
        push((char *) (extraout_XMM0_Qa_06 / local_18));
    }
    break;
case 0x30:
    pcVar3 = (char *)atof(local_88);
    push(pcVar3);
    }
}

pop();
printf("Result: %g\n");
iVar1 = 0;
}
return iVar1;

```

```

while ((type = gettop(s, --argc, ++argv)) != EOF) {
    switch (type) {
case NUMBER:
        push(atof(s));
        break;
case '+':
        push(pop() + pop());
        break;
case '*':
        push(pop() * pop());
        break;
case '-':
        op2 = pop();
        push(pop() - op2);
        break;
case '/':
        op2 = pop();
        if (op2 != 0.0)
            push(pop() / op2);
        else
            printf("error: zero divisor\n");
        break;
default:
        printf("error: unknown command %s\n", s);
        break;
    }
}

```

Ghidra no puede entender parte del programa porque hay errores en las funciones que llaman como en el pop que se supone que devuelve algo, pero esta void en ghidra.

```
int main(int param_1,char **param_2)
```

expr main() arreglado

```
{
    int iVar1;
    double dVar2;
    double dVar3;
    char *__format;
    char **local_98;
    int local_8c;
    char local_88 [112];
    double local_18;
    int local_c;

    local_98 = param_2;
    local_8c = param_1;
    if (param_1 == 1) {
        puts("Usage: expr 1 1 +");
        iVar1 = 1;
    }
    else {
        while( true ) {
            local_98 = local_98 + 1;
            local_8c = local_8c + -1;
            local_c = getop(local_88,local_8c,local_98);
            if (local_c == -1) break;
            switch(local_c) {
                case 0x2a:
                    dVar2 = pop();
                    dVar3 = pop();
                    push(dVar3 * dVar2);
                    break;
                case 0x2b:
                    dVar2 = pop();
                    dVar3 = pop();
                    push(dVar3 + dVar2);
                    break;
                default:
                    printf("error: unknown command %s\n");
                    break;
                case 0x2d:
                    local_18 = pop();
                    dVar2 = pop();
                    push(dVar2 - local_18);
                    break;
                case 0x2f:
                    local_18 = pop();
                    if (local_18 == 0.00000000) {
                        puts("error: zero divisor");
                    }
            }
        }
    }
}
```

```
    }
    else {
        dVar2 = pop();
        push(dVar2 / local_18);
    }
    break;
case 0x30:
    dVar2 = atof(local_88);
    push(dVar2);
}
}
__format = (char *)pop();
printf(__format,"Result: %g\n");
iVar1 = 0;
}
return iVar1;
}
```

```
ulong getop(char *param_1,int param_2,char **param_3)
```

```
{  
    .  
    .  
    .  
}
```

```
int getop(char *s, int argc, char **argv)
```

Ghidra confundio el return value de getop se supone que sea int.

expr getop() arreglado

```
int getop(char *param_1, int param_2, char **param_3)
```

-

```
void pop(void)
{
    if (sp < 1) {
        puts("error: stack empty");
    }
    else {
        sp = sp + -1;
    }
    return;
}
```

```
double pop(void)
{
    if (sp > 0)
        return val[--sp];
    else {
        printf("error: stack empty\n");
        return 0.0;
    }
}
```

Ghidra confudio que pop no devuelve nada.
Pero en el high level codigo devuelve un
numero double.

expr pop() arreglado

```
double pop(void)
{
    double num;

    if (sp < 1) {
        puts("error: stack empty");
        num = 0.00000000;
    }
    else {
        sp = sp + -1;
        num = (double)(&val)[sp];
    }
    return num;
}
```

```

void push(char *param_1)
{
    long lVar1;

    if (sp < 100) {
        lVar1 = (long)sp;
        sp = sp + 1;
        *(char **)(val + lVar1 * 8) = param_1;
    }
    else {
        printf(param_1,"error: stack full, can't push %g\n");
    }
    return;
}

```

```

void push(double f)
{
    if (sp < MAXVAL)
        val[sp++] = f;
    else
        printf("error: stack full, can't push %g\n", f);
}

```

Ghidra no pudo entender que el function push recibe un variable de tipo double. Y se cree que los variables globales sp y val son undefined. En este caso es cambiar el type de sp a int, val a array de double y param_1 a double.

expr push() arreglado

```
void push(double param_1)
{
    int local_EAX_39;

    if (sp < 100) {
        _local_EAX_39 = (long)sp;
        sp = sp + 1;
        *(double *)(&val + _local_EAX_39) = param_1;
    }
    else {
        printf((char *)param_1, "error: stack full, can't push %g\n");
    }
    return;
}
```